

Seguridad y variables

Seguridad en automatizaciones *AI-first*

Las automatizaciones modernas suelen operar mediante múltiples conexiones simultáneas entre plataformas, APIs y servicios externos. **Un mismo *workflow* puede utilizar modelos de IA, bases de datos, herramientas de mensajería y sistemas empresariales.**

Cada conexión incorpora información sensible y nuevos puntos de exposición.

Ejemplo aplicado:

Un *workflow* que analiza consultas con GPT puede requerir simultáneamente:

- API Key de OpenAI.
- credenciales del CRM.
- acceso a correo electrónico.
- conexión con base de datos.

Una falla en cualquiera de estas integraciones puede afectar el funcionamiento completo de la solución.

Principios fundamentales de seguridad

La seguridad en producción **debe pensarse como un proceso continuo**. Requiere definir criterios de trabajo que reduzcan riesgos y faciliten el mantenimiento. Algunos de estos son:

- Evitar guardar credenciales directamente dentro de *workflows*.
- Separar configuraciones entre desarrollo, pruebas y producción.
- Limitar permisos según funciones y responsabilidades.
- Rotar claves periódicamente.
- Utilizar mecanismos seguros para almacenar secretos.
- Monitorear accesos y actividad del sistema.

Variables de entorno

Las variables de entorno son la forma correcta de pasar configuración sensible a las aplicaciones sin escribirla directamente en el código o en los archivos de configuración que podrían compartirse o versionarse. En el contexto de Docker, **se usan para definir credenciales, comportamiento de la aplicación y configuraciones específicas del entorno.**

La práctica recomendada es crear un archivo `.env` en la misma carpeta que el `docker-compose.yml`. Docker Compose lo lee automáticamente:

```
bash
nano .env
```

Dentro del archivo .env:

```
env
DOMAIN_NAME=tu-dominio.com
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=una_contrasena_muy_segura_aqui
N8N_ENCRYPTION_KEY=una_clave_aleatoria_de_32_caracteres
DB_TYPE=postgresdb
DB_POSTGRESDB_HOST=localhost
DB_POSTGRESDB_PORT=5432
DB_POSTGRESDB_DATABASE=n8n
DB_POSTGRESDB_USER=n8n_user
DB_POSTGRESDB_PASSWORD=otra_contrasena_segura
```

Es imprescindible que el archivo .env nunca se suba a un repositorio de Git. Para evitarlo, se agrega al .gitignore:

```
bash
echo ".env" >> .gitignore
```

Autenticación básica y protección del acceso

n8n puede protegerse con autenticación básica, que consiste en un **usuario y contraseña que se solicitan antes de acceder a la interfaz**. Esto es suficiente para un entorno de producción inicial. Para entornos con múltiples usuarios, n8n ofrece un sistema propio que permite invitar colaboradores con distintos niveles de permiso.

Las variables relevantes son

- N8N_BASIC_AUTH_ACTIVE,
- N8N_BASIC_AUTH_USER y
- N8N_BASIC_AUTH_PASSWORD.

Clave de encriptación

La variable `N8N_ENCRYPTION_KEY` es crítica. n8n la utiliza para encriptar todas las credenciales almacenadas (*tokens* de API, contraseñas de cuentas externas, etc.). Si se pierde esta clave y es necesario mover la instancia a otro servidor, las credenciales guardadas quedarán inutilizables y deberán reconfigurarse todas desde cero.

Se puede generar una clave segura con el siguiente comando:

```
bash  
openssl rand -hex 32
```

Esa clave debe guardarse en un lugar seguro fuera del servidor, como un gestor de contraseñas.

Firewall básico con UFW

Ubuntu incluye UFW (*Uncomplicated Firewall*), una **herramienta simple para controlar qué conexiones acepta el servidor**. La configuración mínima recomendada es:

```
bash
# Permitir SSH para no perder acceso
sudo ufw allow OpenSSH

# Permitir tráfico HTTP y HTTPS
sudo ufw allow 80
sudo ufw allow 443

# Activar el firewall
sudo ufw enable
```

Es importante notar que el puerto 5678 no se abre directamente. El acceso a n8n pasa siempre por Nginx en los puertos 80 y 443, lo que agrega una capa de seguridad adicional.

Acceso SSH con claves en lugar de contraseñas

Una práctica de seguridad importante es deshabilitar el acceso SSH por contraseña y usar exclusivamente claves SSH. **Las claves son criptográficamente mucho más seguras que cualquier contraseña.**

En la computadora local, se genera un par de claves si todavía no se tiene uno:

```
bash
ssh-keygen -t ed25519 -C "tu@email.com"
```

Luego se copia la clave pública al servidor:

```
bash
ssh-copy-id root@IP_DEL_SERVIDOR
```

Una vez verificado que el acceso con clave funciona correctamente, es posible deshabilitar el acceso por contraseña editando `/etc/ssh/sshd_config` y estableciendo `PasswordAuthentication no`.

Monitoreo y *logs*

¿Por qué monitorear una automatización?

Una automatización puede ejecutarse correctamente durante las pruebas y presentar problemas una vez que comienza a operar con usuarios reales o integraciones externas.

Algunas situaciones frecuentes:

- APIs que responden lentamente.
- *Workflows* detenidos.
- Errores de autenticación.
- Límites de uso alcanzados.
- Aumento inesperado de volumen.

Indicadores clave para monitorear *workflows*

Rendimiento

- Tiempo de ejecución.
- Cantidad de ejecuciones.
- Uso de recursos.

Operación

- *Workflows* exitosos.
- *Workflows* fallidos.
- Colas pendientes.

Integraciones

- Consumo de APIs.
- Límites de uso.
- Errores externos.

Negocio

- Tiempos de respuesta.
- Volumen procesado.
- Ahorro operativo.

Herramientas habituales de monitoreo

Existen distintas herramientas utilizadas para centralizar información y visualizar el comportamiento de una solución. Algunas herramientas frecuentes:

- **Prometheus:** recopila métricas del sistema y de las ejecuciones, como tiempos de respuesta, cantidad de errores o tareas pendientes.
- **Grafana:** permite visualizar métricas en *dashboards* para detectar tendencias, picos de uso o comportamientos anómalos.
- **Logs de n8n:** registran eventos y errores de ejecución. Ayudan a reconstruir qué ocurrió cuando un *workflow* falla.
- **Alertas:** se generan a partir de reglas definidas sobre *workflows*, *logs* o métricas. Por ejemplo: si un *workflow* falla varias veces, si una API no responde o si una cuota está por agotarse, se puede enviar un aviso por Slack, *email* u otro canal.

Logs de Docker

El comando más útil para ver qué está ocurriendo en la instancia de n8n es:

```
bash
docker compose logs -f n8n
```

La bandera `-f` activa el modo "*follow*" (seguimiento en tiempo real). Muestra cada ejecución de *workflow*, cada error y cada conexión entrante.

Para ver sólo los últimos 100 registros sin seguimiento en tiempo real:

```
bash
docker compose logs --tail=100 n8n
```

Para guardar los *logs* en un archivo y analizarlos posteriormente:

```
bash
docker compose logs n8n > logs-$(date +%Y%m%d).txt
```

Monitoreo desde la interfaz de n8n

n8n incluye una **sección de ejecuciones** en su interfaz **donde se puede ver el historial completo de todos los *workflows***: cuáles tuvieron éxito, cuáles fallaron, cuánto tiempo tardaron y qué datos procesaron. Esta vista es invaluable para el diagnóstico.

Se accede desde la instancia de n8n, dentro del menú lateral, donde dice “*Executions*”.

Configuración de alertas de errores

Una buena práctica es configurar un "Error Workflow" en n8n, es decir, **un workflow especial que se activa automáticamente cuando otro workflow falla**. Este workflow de error puede, por ejemplo, enviar un mensaje de Telegram o Slack con el detalle del error, el nombre del workflow fallido y la hora en que ocurrió. Esto permite conocer los problemas en tiempo real sin necesidad de revisar los logs manualmente.

Para configurarlo, se accede a n8n, *Settings*, *Workflow Settings* y *Error Workflow*. Ahí se selecciona el workflow que debe ejecutarse cuando ocurra un error.

Herramientas externas de monitoreo

Para un monitoreo más robusto, existen herramientas gratuitas como [UptimeRobot](#) que **verifican cada pocos minutos si n8n responde correctamente y envían una alerta por correo o Telegram si el servicio cae**. La configuración toma menos de cinco minutos y ofrece visibilidad del estado del servicio tanto al equipo técnico como a los clientes.